FLUTTER COURSE · SESSION 6 OF 15

Working with APIs

Until now, every piece of data in our apps was typed into the code. Today the data comes from the internet: you'll ask a server for it, turn the answer into Dart objects, and show the right screen while you wait, when it fails, and when there's nothing to show.

BY THE END OF TODAY YOU CAN

Fetch data from a web API with `http`, turn the JSON into typed model classes with `fromJson`, and show loading, error, empty and data states, including no internet and timeouts.

Setup

```
flutter pub add http
```

In VS Code, today's apps are in the **Session 6** group of the Run and Debug dropdown. Before you write any code, open these addresses in your browser and look at the data:

- `https://jsonplaceholder.typicode.com/users`: 10 sample users (demos)
- `https://official-joke-api.appspot.com/random_joke`: a random joke (project). Refresh the page and you get a new one.

RIVERPOD 3 NOTE

When a provider fails, Riverpod 3 quietly retries it for about 30 seconds while showing the spinner. Our apps turn that off so the error message appears straight away: `ProviderScope(retry: (retryCount, error) => null, child: ...)`.

The big picture: the waiter

An **API** is like a waiter. Your app doesn't walk into the kitchen (the server's database). It gives the waiter an order (a **request** to an address). The waiter comes back with a short answer (“here you go”, “we don't have that”, “the kitchen is on fire”) and a plate

of data. The plate arrives wrapped as one long piece of text, so we unwrap it and put

1 · request

Your app asks an address

`http.get(url)`

→

2 · wait

Takes time. Show a spinner

`await`

→

3 · response

A status code and a body (text)

`200`

.

`"{…}"`

→

4 · decode

Text → Dart Map or List

`jsonDecode`

→

5 · model

Map → typed object

`User.fromJson`

API words, in plain English

WORD	WHAT IT MEANS
API	A web address that returns data instead of a web page.
Endpoint / URL	The exact address you ask, e.g. `…/users/1`. Different addresses give different data.
Request	Your app's question. **GET** reads data, **POST** creates, **PUT/PATCH** updates, **DELETE** removes. Today we only use GET.
Status code	A 3-digit answer to “did it work?”. 200 means yes.
Body	The data in the response, always one long String.
JSON	The text format of the data. `{ }` is one object (becomes a `Map`), `[ ]` is a list (becomes a `List`).
`jsonDecode`	Turns the JSON text into Dart Maps and Lists. Comes from `dart:convert`.
Model class	A Dart class for one item, e.g. `User` with `name` and `email`.
`fromJson`	A constructor that builds one model object from one Map. Lets you write `user.name` instead of `json['name']`.
`Future`, `async`, `await`	From Session 1: a value that arrives later. `await` waits for it without freezing the app.
`.timeout(…)`	Give up after a set time instead of spinning forever.
`FutureBuilder`	A widget that shows one thing while a Future is waiting and another when it's done.
`FutureProvider`	The Riverpod way: a provider that runs an async function and remembers the result.
`AsyncValue`	What a FutureProvider gives you: **loading**, **error** or **data**, never more than one at a time.
`.when(…)`	Asks you for one widget per state: loading, error, data.

`ref.invalidate(p)`	Throw away the old result and fetch again. That's how Retry and “Next joke” work.

Status codes you need to know

CODE	MEANS	WHAT YOUR APP SHOULD DO
200	OK, here's the data	Decode and show it
201	Created (after a POST)	Show success
400	Bad request: you sent something wrong	Fix the request
401	Not logged in	Send the user to login
404	Not found	Show “not found”
500	The server broke	Show an error with Retry

The concepts, in the order we cover them

`demo1` http.get, status codes and jsonDecode

Two buttons: one asks for user 1 (status **200**), the other for user 9999, who doesn't exist (status **404**). The screen shows the raw body first, then the same data after `jsonDecode`.

```
import 'dart:convert';
import 'package:http/http.dart' as http;

final response = await
http.get(Uri.parse('https://jsonplaceholder.typicode.com/users/1'));
response.statusCode   // 200
response.body         // '{"id":1,"name":"Leanne Graham",...}'  ← one long String

final Map<String, dynamic> json = jsonDecode(response.body);
json['name']                 // 'Leanne Graham'
json['address']['city']      // nested: one Map inside another
```

`model` Model class with fromJson

Reading `json['name']` everywhere is risky: one typo and the app crashes at runtime.

A model class reads the JSON **once**, in one place.

```
// what the API sends (one user)
{
  "id": 1,
  "name": "Leanne Graham",
  "email": "Sincere@april.biz",
  "company": {
    "name": "Romaguera-Crona"
  }
}
```

```
class User {
  ...
  factory User.fromJson(
      Map<String, dynamic> json) {
    return User(
      id: json['id'] as int,
      name: json['name'] as String,
      email: json['email'] as String,
      company: json['company']['name']
          as String,
    );
  }
}
```

The API sends a **list** of users, so decode it as a List and turn each Map into a User:

```
final List<dynamic> data = jsonDecode(response.body);
return data.map((item) => User.fromJson(item)).toList();
```

`errors` Turning every failure into a sentence

Four things can go wrong. Catch each one and throw a message a user understands.

This is in `users_api.dart`:

```
try {
  final response = await http.get(url).timeout(const Duration(seconds: 10));
  if (response.statusCode != 200) {
    throw ApiException('The server answered ${response.statusCode}. Please try
again.');
  }
  ...
} on TimeoutException {          // took longer than 10 seconds
  throw ApiException('The server took too long to answer...');
} on http.ClientException {      // no Wi-Fi / no data / server unreachable
  throw ApiException('No internet connection...');
} on FormatException {           // the body wasn't valid JSON
  throw ApiException('The server sent data we could not read.');
}
```

---

states The four states every loading screen needs

loading

Waiting for the server. Show a spinner.



error

Say what went wrong in plain words, and offer Retry.



empty

It worked, but there's nothing. Say so kindly.



data

Show the list.

Demos 2 and 3 have **Break** and **Empty** switches in the AppBar, so you can see the error and empty states without switching off the Wi-Fi.

---

demo2 FutureBuilder

```
late Future<List<User>> _usersFuture;

@override
void initState() {
  super.initState();
  _usersFuture = fetchUsers();     // start the request ONCE, here
}

FutureBuilder<List<User>>(
  future: _usersFuture,
  builder: (context, snapshot) {
    if (snapshot.connectionState == ConnectionState.waiting) return Spinner();   //
loading
    if (snapshot.hasError) return ErrorView(snapshot.error);
// error
    final users = snapshot.data ?? [];
    if (users.isEmpty) return Text('No users found.');                           //
empty
    return ListView.builder(...);                                                //
data
  },
)
```

Never call `fetchUsers()` inside `build()`. `build` runs many times, and each run would send a new request. Start it in `initState` and keep the Future in a field. Retry calls `setState` with a new Future.

---

demo3 FutureProvider and AsyncValue

The same screen, the Riverpod way. No StatefulWidget, no `initState`, no Future in a field: the provider remembers the result.

```
final usersProvider = FutureProvider<List<User>>((ref) {
  return fetchUsers();
});

// in a ConsumerWidget:
final usersAsync = ref.watch(usersProvider);     // AsyncValue: loading OR error OR data

return usersAsync.when(
  loading: () => Spinner(),
  error: (error, stackTrace) => ErrorView(
    message: error.toString(),
    onRetry: () => ref.invalidate(usersProvider),   // fetch again
  ),
  data: (users) => users.isEmpty ? Text('No users found.') : ListView.builder(...),
);
```

- `.when` makes you write all three. Forget one and it won't compile.
- `ref.invalidate(usersProvider)` throws away the old result and fetches again.
- Pull to refresh: `RefreshIndicator(onRefresh: () => ref.refresh(usersProvider.future), ...)`.
- The provider **watches** the Break and Empty switches, so flipping a switch fetches again by itself.

	FUTUREBUILDER (DEMO 2)	FUTUREPROVIDER (DEMO 3)
Where the Future lives	A field in a State class	The provider
Start the request	`initState`	Automatically, the first time it's watched
Check the state	`if` on `snapshot`	`.when(loading, error, data)`
Retry	`setState` with a new Future	`ref.invalidate(provider)`
Use the data on another screen	Pass it through constructors	Just `ref.watch` the same provider

The project: Joke App

One screen with a joke in the middle: the setup, then the punchline. While a joke loads you see a spinner. If it fails you see a clear message, and the button becomes “Try again”. Tap **Next joke** for a new one.

```
lib/joke_app/
├─ main.dart · ProviderScope with retry turned off
├─ models/joke.dart · Joke.fromJson
├─ services/joke_service.dart · all network code and error messages
├─ providers/joke_providers.dart · httpClient → service → jokeProvider
└─ screens/joke_screen.dart · .when + Next joke
```

FILE	WHAT TO LOOK FOR
`joke_service.dart`	The four error types turned into readable messages. The `http.Client` is passed in, so tests can give it a fake server.
`joke_providers.dart`	Three providers in a chain: `httpClientProvider` → `jokeServiceProvider` → `jokeProvider` (a FutureProvider).
`joke_screen.dart`	`.when` for the three states. Next joke is `ref.invalidate(jokeProvider)`, and the button is disabled while loading.
`test/joke_app_test.dart`	A fake server (`MockClient`) tests a working joke, a 500 error with retry, and no internet, without touching the real internet.

Errors you will see today

> **`Spinner forever, and the request is sent again and again`**
>
> You called the fetch function inside `build()`. Start it in `initState`, or use a FutureProvider.

> **`"type 'List<dynamic>' is not a subtype of type 'Map<String, dynamic>'"`**
>
> The API returned a list `[...]` and you treated it as one object `{...}`, or the other way round. Open the URL in a browser and check the first character.

> **`"type 'Null' is not a subtype of type 'String'" inside fromJson`**
>
> A key is misspelled, or the value is nested one level deeper (like `company.name`).

> **`"ClientException: Failed to fetch" in Chrome`**
>
> Wrong address, `http://` instead of `https://`, or an API that doesn't allow browsers. Today's APIs all work in Chrome.

> **`The error screen takes about 30 seconds to appear`**
>
> Riverpod 3's automatic retry. Add `retry: (retryCount, error) => null` to `ProviderScope`.

Practice task (in class)

- [ ] Fetch posts from `https://jsonplaceholder.typicode.com/posts`.
- [ ] Parse them into a `Post` model with `id`, `title` and `body`, using `fromJson`.
- [ ] Show them in a `ListView.builder`.
- [ ] Handle all four states: loading spinner, error message with Retry, empty, data.
- [ ] Bonus: tap a post to open a detail screen.

Homework: extend the Joke App

- [ ] Hide the punchline until the user taps **Show punchline**.
- [ ] Add a "Programming jokes only" switch using `https://official-joke-api.appspot.com/jokes/programming/random`. Careful: this one returns a **list** with one joke inside.
- [ ] Show "Joke #id" on the card.
- [ ] Commit your work and push it to GitHub.

Check yourself

▼ **What do status codes 200, 404 and 500 mean?**

200: it worked. 404: not found. 500: the server broke.

▼ **Why must the fetch not be called inside build()?**

build() runs many times. Each run would send a new request, and the screen would keep reloading.

▼ **What are the four states of a loading screen?**

Loading, error, empty and data.

▼ **What does fromJson do, and what happens if a key is misspelled?**

It builds one model object from one JSON Map. A misspelled key gives null, so the app crashes at runtime with a “Null is not a subtype” error.

▼ **What does ref.invalidate(jokeProvider) do?**

It throws away the current result and runs the provider again, which fetches a new joke.

NEXT SESSION

Session 7 · Local Storage + Checkpoint 1. Saving data on the phone with `shared_preferences` so it survives a restart, then your first checkpoint project.